User Manual

1. Software Used:

Frontend (Website Design)

- HTML/CSS
- ReactJS

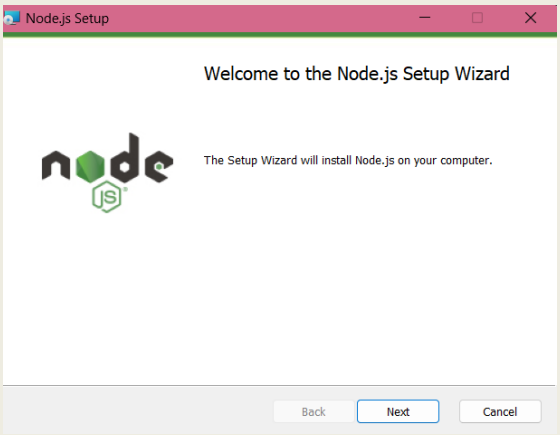
Backend (Application Source Code & Logic)

- JavaScript
- Python
- Azure Speech & Translator APIs

2. Setup Instructions

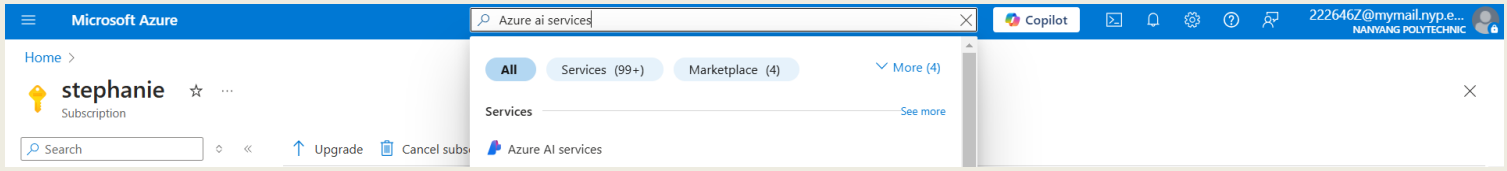
Step 1: Installing NodeJS

- Install NodeJS from their official website.
- Once installing the extension, you should see this. Click next until the finish installation button and do not edit any of the system requirements.

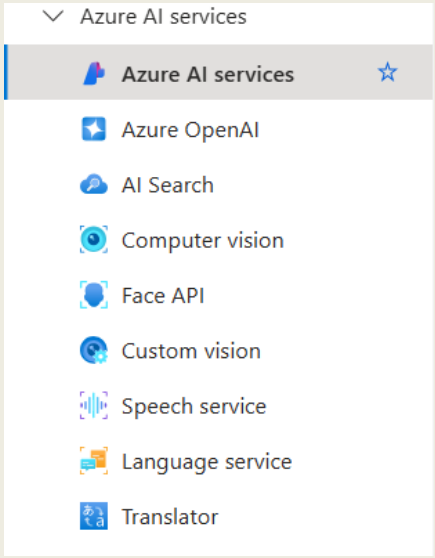


Step 2: Deploy Speech & Translator Models from Azure

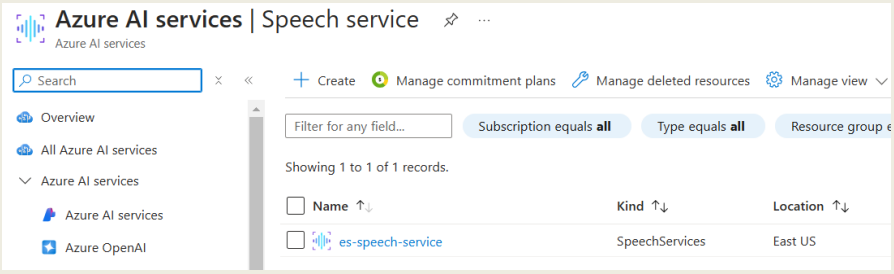
- Create an azure account for free.
- In the search bar on top, type in “Azure AI Service”. Select It.



- Select speech & translator service.



- Click Create to customize & deploy model.



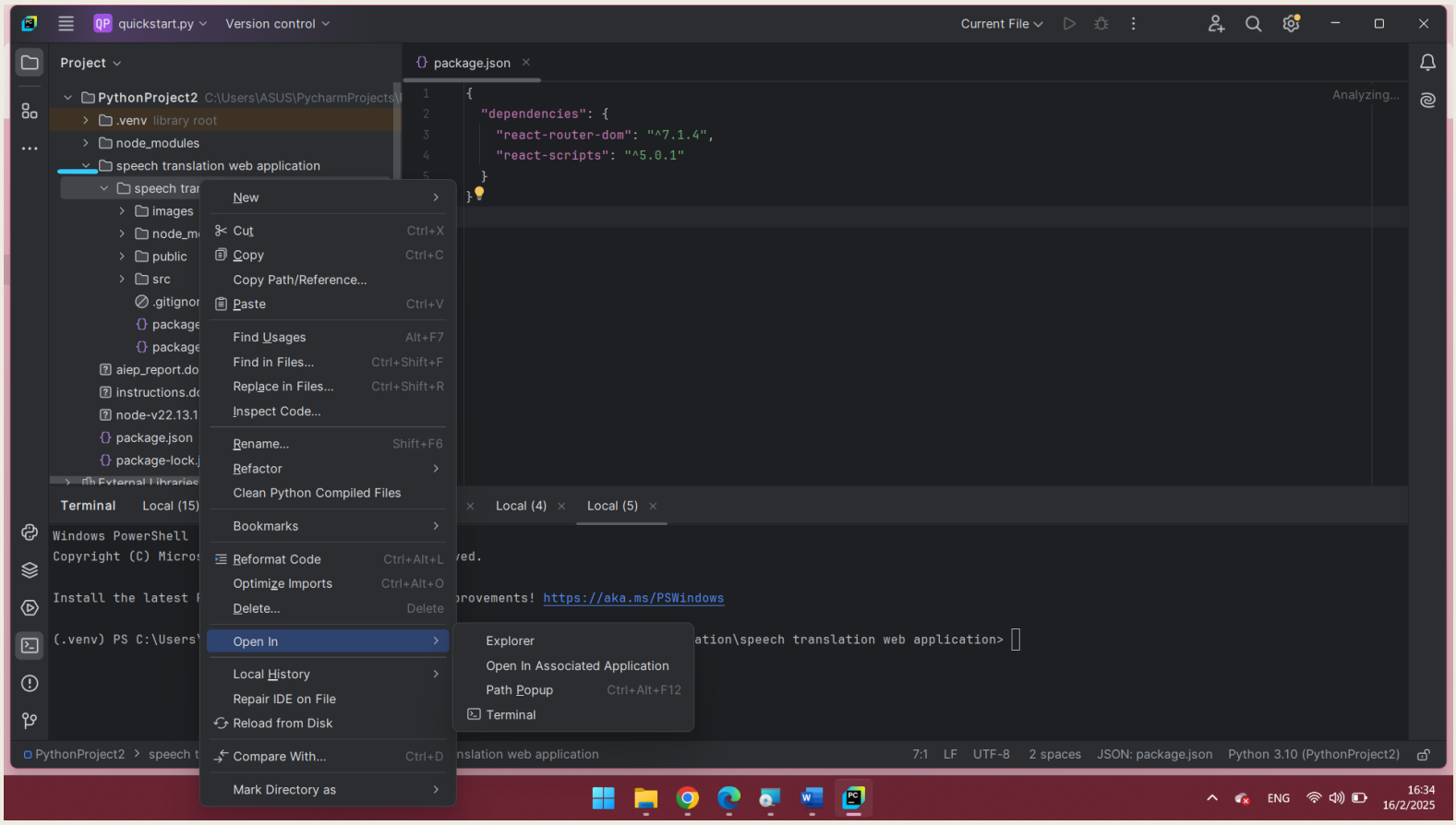
- Customize the names to a unique one. Then click review + create. Deployment will take a few minutes.

- Do the same for the translator service.

- Once done, collect their keys & endpoints.

Step 3:

- Download the entire project from GitHub.
- Upload this code on pycharm.
- Create your own virtual environment: [click me to find out how]
- Click the speech translation web application file. Right click and select open in terminal.



- In the terminal, enter these few lines of code:

  → npm install

```
(.venv) PS C:\Users\ASUS\PycharmProjects\PythonProject2\speech translation web application\speech translation web application> npm install
```

- Got to TranslatorApp.js (under src/components/)

```
1  import React, { useState } from "react";
2  import {
3    SpeechConfig,
4    AudioConfig,
5    SpeechRecognizer,
6    SpeechSynthesizer,
7  } from "microsoft-cognitiveservices-speech-sdk";
8
9  const TranslatorApp = () => {
10   const [inputText, setInputText] = useState("");
11   const [translatedText, setTranslatedText] = useState("");
12   const [targetLanguage, setTargetLanguage] = useState("zh-CN"); // Default to Chinese for output
13   const [inputLanguage, setInputLanguage] = useState("en-SG"); // Default to English for input
14   const [isTranslating, setIsTranslating] = useState(false);
15   const [isRecording, setIsRecording] = useState(false); // State for recording status
16
17   // Azure Keys (to edit)
18   const speechKey = "CWfB8JF6FYq30EqMGkfz1l7Sqj28OWQNsj9coQ8YKIDzFqLi4L0oJQQJ99BAACYeBjFXJ3w3AAAYACOGkQ8H";
19   const speechRegion = "eastus";
20   const translatorKey = "FDmJB51751N0FfF6r9E3QDseFTJd2jfiI6kbkDyqivdoCVrCKZY6JQQJ99BAACYeBjFXJ3w3AAAbACOG2MV7";
21   const translatorEndpoint = "https://api.cognitive.microsofttranslator.com/";
22
23   let recognizer; // Store recognizer instance to stop recording
24
25   // Start Recording
26   const handleStartRecording = async () => {
27     setIsRecording(true);
28     const speechConfig = SpeechConfig.fromSubscription(speechKey, speechRegion);
29     speechConfig.speechRecognitionLanguage = inputLanguage; // Dynamically set the input language
30
31     const audioConfig = AudioConfig.fromDefaultMicrophoneInput();
32     recognizer = new SpeechRecognizer(speechConfig, audioConfig);
33
34     recognizer.recognizeOnceAsync(
35       (result) => {
36         setInputText(result.text);
37         recognizer.close();
```

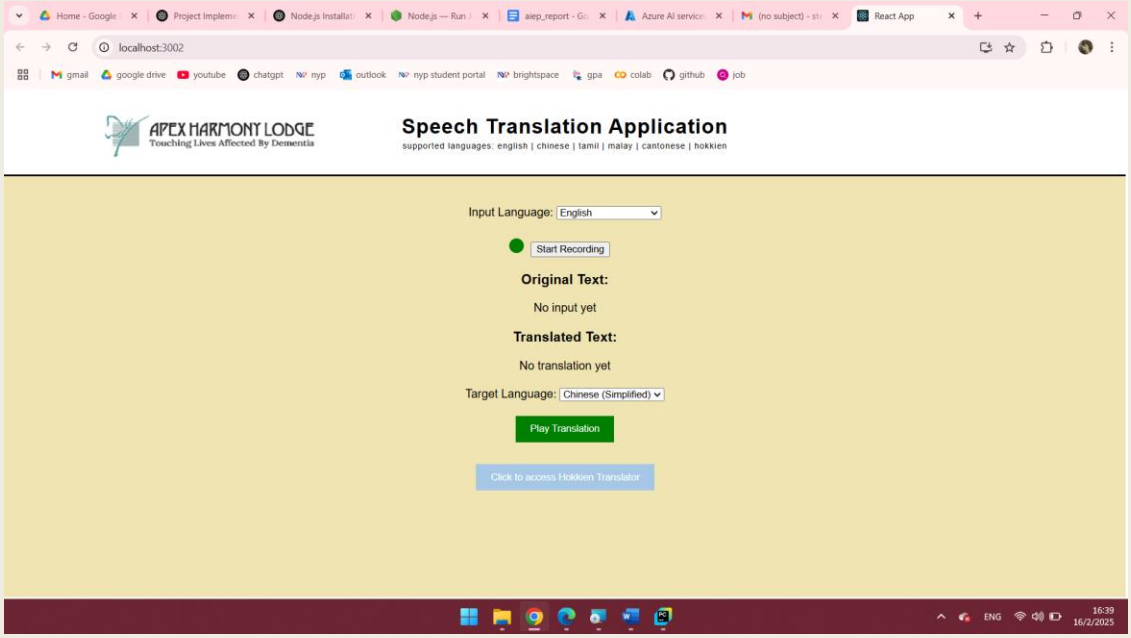
- Edit lines 18-21 with your Azure keys and endpoint saved from Step 2

- Then enter

  → npm start

- This should start up the web application. Click on the local link.

- You have now successfully accessed the web application.



!! Notes:

- TranslatorApp.js is the main source code behind the Application's logic. In this file, users add their Azure endpoints, keys, and select the languages they want to translate in the application.
- app.css stores information of the design of website.